

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»
ITMO University

Факультет Прикладной информатики

Программирование в инфокоммуникационных системах

ОТЧЕТ

о практике «Учебная, ознакомительная практика»

Тема задания: Веб-приложение для управления игровыми сессиями D&D с системой приоритетов и балансировки групп.

Обучающийся: Сакулин Иван Михайлович К3221,
Сафронов Иван Сергеевич К3221, Мануковская Дарья Михайловна К3221,
Пухарев Сергей Валерьевич К3220, Мартенс Дмитрий Павлович К3220.

Согласовано:

Руководитель практики от университета: Казанова Полина Петровна,
практикант, факультет Прикладной Информатики

Практика пройдена с оценкой _____

Дата _____

Санкт-Петербург

2026

СОДЕРЖАНИЕ

Стр.

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ ..	3
ВВЕДЕНИЕ	4
1 ВЫПОЛНЕНИЕ РАБОТЫ	5
1.1 Анализ предметной области	5
1.1.1 Аналогии.....	6
1.1.2 Проблемы, решаемые продуктом	8
1.2 Разработка проекта программной системы.....	9
1.2.1 Требования к системе (Дари Мануковская)	9
1.2.2 Бизнес-процессы системы (Иван Сакулин)	12
1.2.3 Используемые технологии разработки (Иван Сакулин) ...	15
1.2.4 Диаграмма классов и их связь (Сергей Пухарев)	18
1.2.5 Диаграммы последовательности, деятельности и состояния (Сергей Пухарев)	20
1.2.6 Схема базы данных (Иван Сафронов)	22
1.2.7 Дизайн-прототип (Дмитрий Мартенс)	24
1.3 Выводы.....	31
ЗАКЛЮЧЕНИЕ	32
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	33

СПИСОК СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

D&D – ролевая настольная игра Dungeons & Dragons

ВВЕДЕНИЕ

В условиях активного развития веб-технологий и роста популярности настольных ролевых игр актуальной становится задача автоматизации процессов организации и управления игровыми сессиями. Одной из наиболее распространённых систем является Dungeons & Dragons, требующая координации игроков, мастеров, расписаний, а также соблюдения баланса игровых групп. Отсутствие специализированных инструментов управления приводит к перегрузке ведущих, снижению качества игрового процесса и неэффективному распределению участников по сессиям.

Необходимо спроектировать веб-приложение, обеспечивающее централизованное управление игровыми сессиями D&D с использованием системы приоритетов и механизмов балансировки групп.

Цель: проектирование веб-приложения для управления игровыми сессиями D&D с поддержкой приоритетного распределения участников и балансировки игровых групп.

Задачи:

- 1) анализ предметной области и существующих решений для управления игровыми сессиями;
- 2) формализация функциональных и нефункциональных требований;
- 3) моделирование ключевых бизнес-процессов системы;
- 4) разработка базы данных;
- 5) создание прототипа интерфейса веб-приложения.

В качестве планируемых результатов практики предусматривается получение формализованного описания системы для управления игровыми сессиями D&D с системой приоритетов и балансировки групп, которое может быть использовано в дальнейшем при реализации программного продукта. Итоговые материалы могут служить основой для разработки прототипа или полноценной информационной системы.

1 ВЫПОЛНЕНИЕ РАБОТЫ

В этом разделе представлены результаты выполнения работы.

1.1 Анализ предметной области

Первым шагом практики стал анализ предметной области, компании и её требований, затем было проведено исследование аналогов платформы. Компанией-заказчиком может выступать организация, проводящая партии D&D или похожие игры.

Конкретный пример такой компании: подразделение ролевых игр D&D клуба настольных игр «GEEKMO». Запись на игры в изучаемой компании проводится с помощью различных сторонних инструментов (публикации в социальной сети «VK», записи через «google»-таблицы). Отбор и бизнес-логика выполняются вручную, отсутствует прозрачность обновления данных в таблицах для регистрации на игры.

Примерное число сотрудников компании (организаторы игр) в разное время составляет от 15 до 20 человек, только от двух до пяти из них используют персональный компьютер для организации процесса и модерации заявок. На администраторской роли был четко обозначен лишь один сотрудник. Отсюда следует, что размер компании можно определить как «очень малая».

Определены основные типы пользователей: помимо разработчиков и администраторов были также выделены обычные пользователи (игроки и игровые мастера) и модераторы (обычные пользователи с правом блокировки нежелательного контента).

На разработку веб-приложения было решено выделить однократные вложения в бюджет. После анализа была выбрана итерационная модель жизненного цикла (поэтапная разработка до достижения готового результата). Поддержку после создания платформы будет осуществлять обученный администратор компании. Найденные в течении определённого времени после окончания разработки во время эксплуатации ошибки остаются на исправления разработчикам.

1.1.1 Аналоги

В качестве аналогов продукта были найдены и рассмотрены Fantasy Grounds LFG, RPGTableFinder и Discord.

Все три платформы обеспечивают функциональность анонсирования и записи на партии, однако каждая обладает существенными ограничениями. Fantasy Grounds LFG представляет собой встроенное решение, но в рамках платной экосистемы: запись осуществляется через комментарии и очерёдность составляется в зависимости от времени записи в хронологическом порядке, отсутствуют средства фильтрации и автоматизации отбора, что делает инструмент пригодным исключительно для существующих групп, уже инвестировавших в лицензирование платформы [1]. Discord предлагает бесплатную и гибкую среду, но ценой полной децентрализации: анонсы быстро теряются в потоках сообщений, процесс записи сводится к гонке за реакциями, а модерация очередей и обработка заявок ложится на мастера в ручном режиме. RPGTableFinder является профильным сервисом с структурированным каталогом партий и системой подачи заявок, однако его русскоязычная аудитория остаётся малочисленной, а механизм отбора полностью делегирован мастеру без каких-либо алгоритмов прозрачного или равномерного распределения мест [2].

Настоящее исследование позволяет идентифицировать системный дефицит на рынке платформ для организации игровых сессий в настольных ролевых играх, который выражается в отсутствии механизмов, гарантирующих уравнительный доступ к игровым ресурсам. Анализ существующих решений (таблица 1) демонстрирует, что их функциональность различается, однако при этом критически важный аспект – алгоритмическое обеспечение справедливого распределения игровых мест – остаётся нереализованным.

Таблица 1 — Сравнение с аналогами

Название	Преимущества	Недостатки
Fantasy Grounds LFG	1) Прямая интеграция с виртуальным столом Fantasy Grounds; 2) целевая аудитория опытных игроков и мастеров; 3) поддержка множества игровых систем (D&D, Pathfinder, Savage Worlds)	1) Требуется покупка лицензии (финансовый барьер); 2) устаревший форумный интерфейс, нет удобных фильтров; 3) отсутствие инструментов для справедливого отбора игроков
Discord	1) Гибкость и бесплатность настройки серверов под сообщества; 2) мгновенная коммуникация через голос и чаты; 3) огромная и активная аудитория игроков	1) Полный хаос в организации записи («гонка за места»); 2) отсутствие единого каталога и профилей игроков; 3) ручная работа модераторов по управлению списками
RPGTableFinder	1) Специализированный сервис для поиска онлайн и оффлайн игр; 2) возможность организовать свой стол и управлять расписанием; 3) удобный интерфейс с поиском и фильтрами	1) Небольшая база пользователей, особенно русскоязычных; 2) нет интеграции с популярными виртуальными столами; 3) нет алгоритмов для справедливого распределения мест

1.1.2 Проблемы, решаемые продуктом

Разрабатываемая платформа отличается от большинства других сервисов для поиска игр D&D. Её основная идея заключается в справедливом распределении игроков, так, чтобы возможность попасть в игру была у каждого. Приоритет получает тот пользователь, который дольше не принимал участие в играх. Данный подход решает несколько проблем, которые имеются у уже существующих аналогов:

- 1) Исключение случайности – у каждого пользователя одинаковые возможности для попадания на игру, причём каждый пользователь понимает принцип отбора.
- 2) Меньше ручной работы – разрабатываемая платформа сама формирует состав игроков, рассылает уведомления и ведёт очередь. Мастеру не нужно договариваться с каждым участником лично.
- 3) Более устойчивое сообщество – благодаря системе приоритетов, у каждого пользователя равные шансы попасть в игру, что повышает шансы новых пользователей остаться в сообществе.
- 4) Прозрачные правила – все решения принимаются по заранее известным для всех пользователей правилам. Пользователи понимают, почему именно они получили или не получили место, и видят своё положение в очереди. Это снижает количество конфликтов и повышает доверие к платформе.

Таким образом, главное отличие разрабатываемой платформы в том, что она не просто соединяет людей, а активно и справедливо управляет распределением игровых сессий. Это помогает решить проблему ограниченного числа мест в играх при большом количестве желающих и предлагает понятные и честные правила взаимодействия внутри игрового сообщества.

1.2 Разработка проекта программной системы

В данном разделе представлено проектирование программной системы, включающее разработку архитектуры веб-приложения, моделирование бизнес-процессов, проектирование базы данных и определение используемых технологий.

1.2.1 Требования к системе (Дари Мануковская)

В данном разделе формализуются требования к веб-приложению для управления игровыми сессиями D&D с системой приоритетов и балансировки групп. Требования определяют функциональные возможности системы, её качественные характеристики, а также границы проекта.

Бизнес-требования

Система предназначена для централизованного управления игровыми сессиями D&D и автоматизации процесса формирования игровых групп. Основные цели:

- оптимизация записи игроков на сессии;
- снижение организационной нагрузки на мастеров;
- прозрачное распределение игроков по сессиям;
- учет приоритетов и активности участников;
- обеспечение балансировки групп.

Функциональные требования

Система должна управлять пользователями:

- регистрировать и предоставлять доступ;
- создавать, редактировать, удалять персонажей;
- предоставлять инструменты модерации персонажей;

- предоставлять возможности управления профилем и просмотра истории участия.

Система должна управлять игровыми партиями:

- создавать, редактировать, удалять игры;
- предоставлять инструменты модерации игр;
- публиковать игры;
- обеспечивать задание параметров (дата, лимит игроков, требования);
- обеспечивать запись игроков и управление составом группы;
- загружать ленту доступных для записи опубликованных игр.

Система должна обеспечивать приоритезацию и балансировку групп:

- хранение и расчет приоритетов игроков;
- автоматическое формирование групп по заданным критериям (уровень, опыт, роли).

Нефункциональные требования

- клиент–серверная архитектура;
- хранение данных в централизованной базе данных;
- разграничение доступа по ролям;
- защита данных и базовая безопасность веб-приложения;
- время отклика до двух-трёх секунд при стандартной нагрузке.

Границы проекта

Входит в состав проекта:

- проектирование архитектуры системы;
- моделирование бизнес-процессов;
- формализация требований;
- разработка концепции алгоритма приоритетов и балансировки;
- описание структуры данных и компонентов системы;
- подготовка диаграмм;
- разработка прототипа интерфейса;

- разработка прототипа системы.

Не входит в состав проекта:

- реализация игровой механики D&D;
- редактор изображения персонажа;
- разработка боевого движка или автоматического расчёта характеристик;
- интеграция с другими платформами;
- коммерциализация;
- форумы и социальные ленты.

Диаграмма прецедентов

Для формализации взаимодействия пользователей с системой и визуализации функциональных возможностей была построена диаграмма прецедентов в нотации UML в соответствии с правилами самоучителя [3]. Диаграмма отражает основные роли пользователей и сценарии их взаимодействия с веб-приложением (рисунок 1).

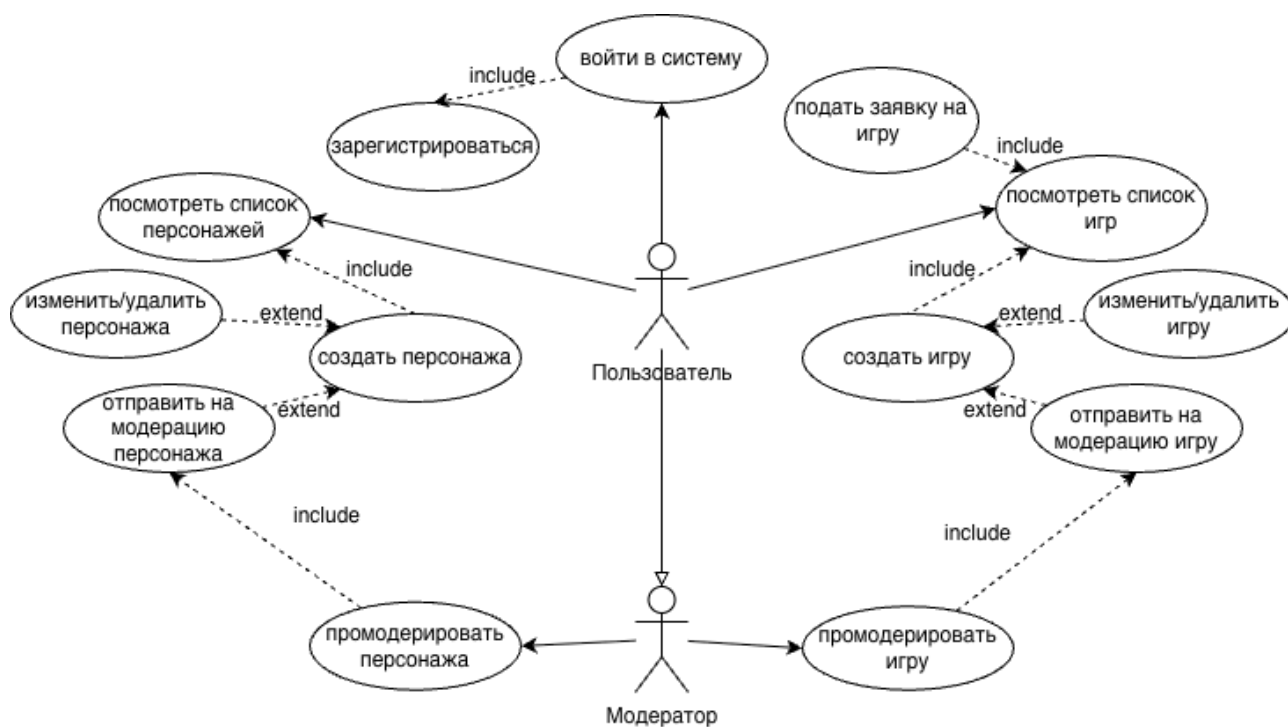


Рисунок 1 — Диаграмма-прецедентов

1.2.2 Бизнес-процессы системы (Иван Сакулин)

Для описания процессов была использована нотация BPMN.

Процесс входа в систему представлен на рисунке 2. Для прохождения этого процесса необходимы регистрационные данные: логин и пароль.

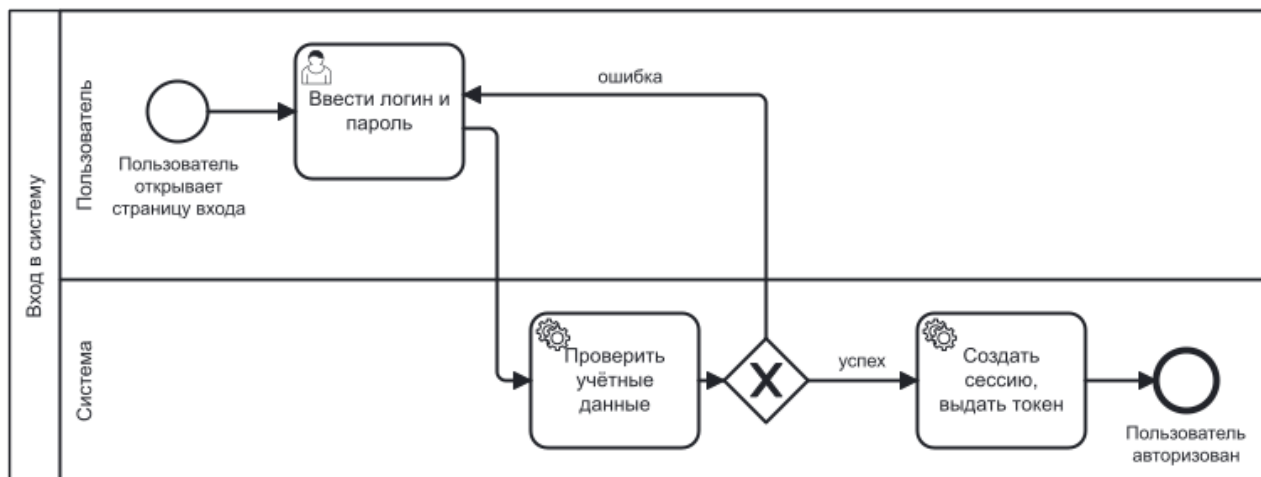


Рисунок 2 — Вход в систему

Если пользователь ещё не зарегистрирован в системе, ему предоставляется возможность пройти процесс регистрации (3).

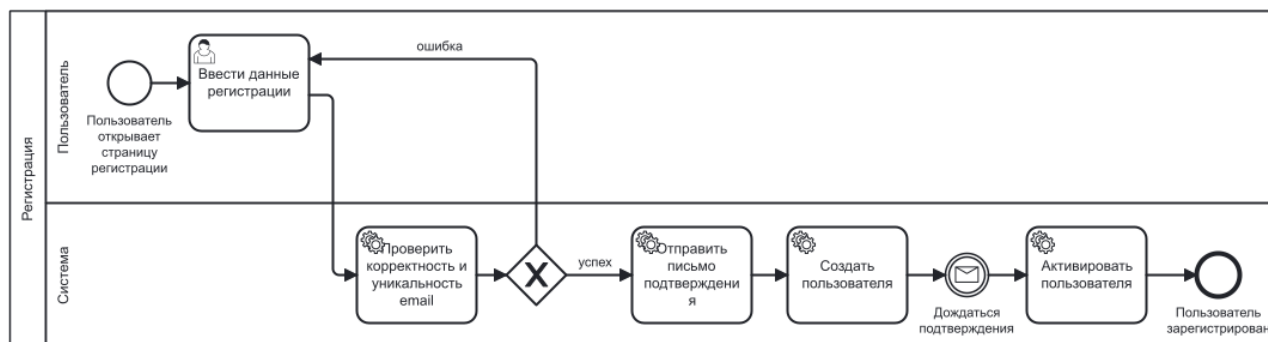


Рисунок 3 — Регистрация в системе

Игра (партия) организуется и проводится игровым мастером. Чтобы система выполняла задачи распределения игроков, мастера должны создавать и публиковать свои игры на платформе, этот процесс отражён на рисунке 4.

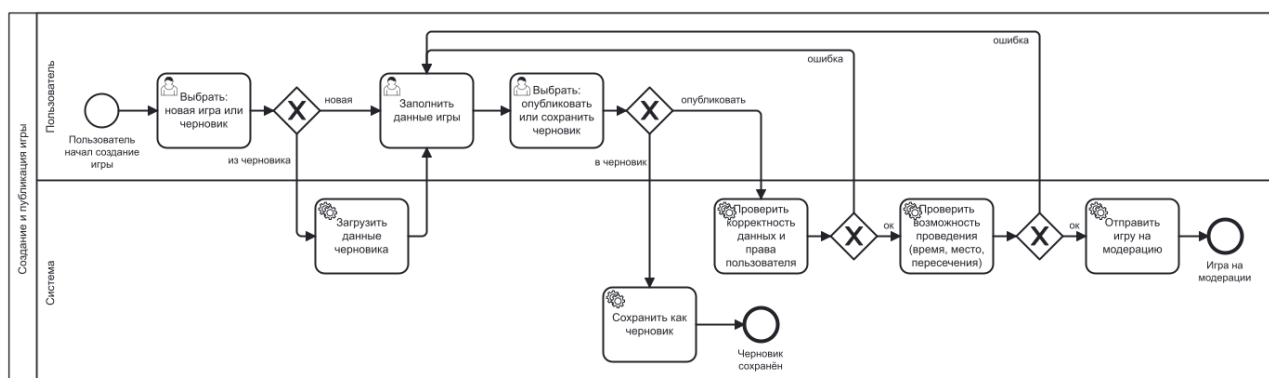


Рисунок 4 — Создание и публикация игры

Одним из важнейших аспектов D&D является игровой персонаж, чтобы игра состоялась, игрок должен продумать его характеристики и корректно задать его (рисунок 5).

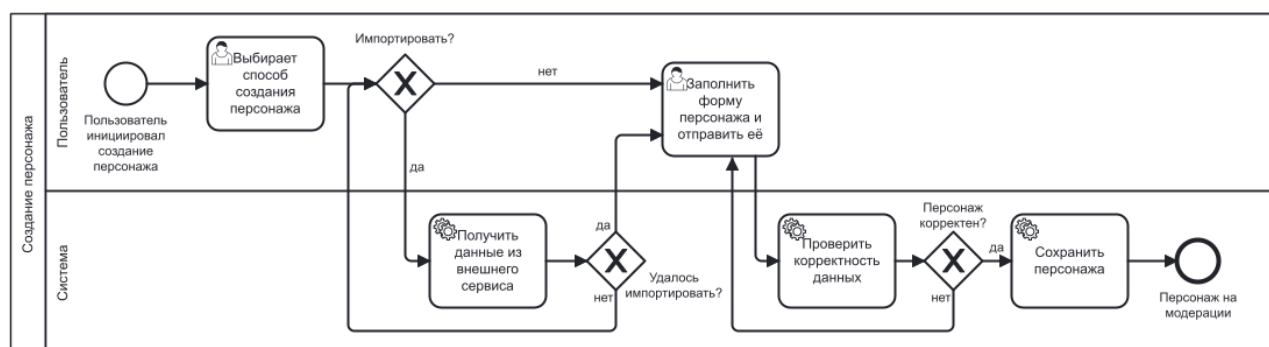


Рисунок 5 — Создание персонажа

Ключевая функция приложения – регистрация игрока на игру, её процесс также подробно описан на рисунке 6.

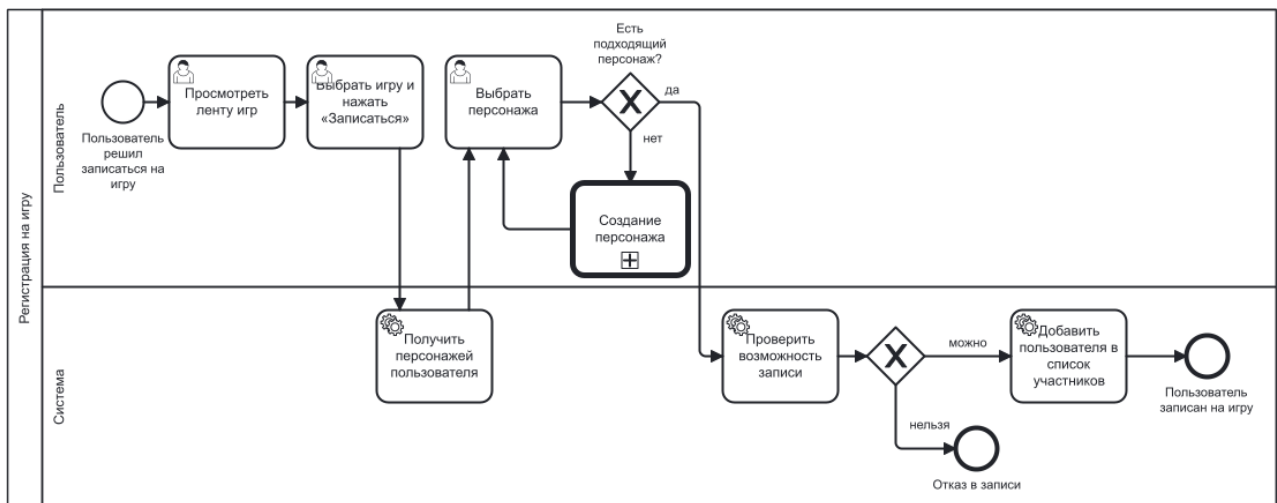


Рисунок 6 — Регистрация на игру

Для веб-приложения с публикациями необходима модерация, чтобы устранить нецензурный или запрещённый контент. В качестве основной была выбрана модель премодерации: модераторы периодически или при поступлении жалобы проверяют игры, отправленные на модерацию (рисунок 7).

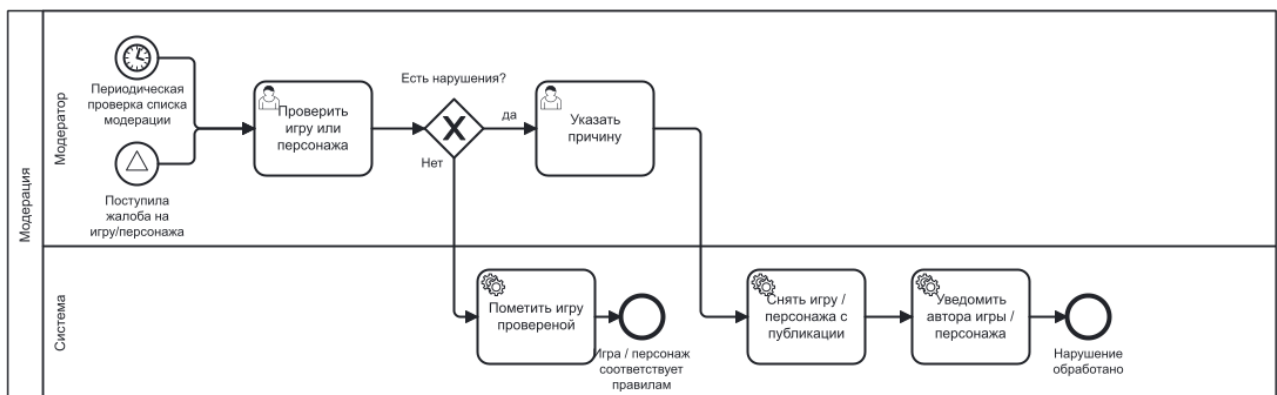


Рисунок 7 — Модерация игр

По окончании набора игроков происходит процесс, завершающий жизненный цикл партии в веб-приложении (рисунок 8).

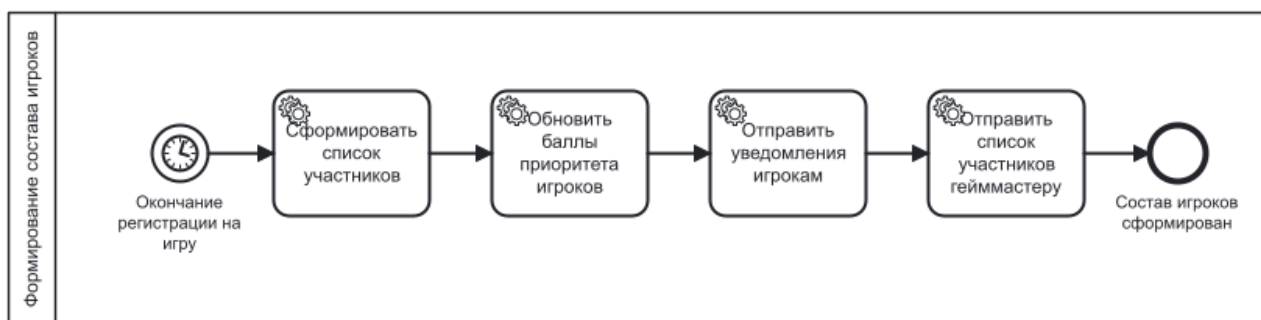


Рисунок 8 — Окончание регистрации

1.2.3 Используемые технологии разработки (Иван Сакулин)

При разработке веб-приложения была выбрана клиент-серверная архитектура, обеспечивающая разделение пользовательского интерфейса, серверной логики и уровня хранения данных. Выбор конкретных технологических решений для каждого уровня осуществлялся на основе сравнительного анализа существующих альтернатив с учетом требований к производительности, масштабируемости и скорости разработки.

Выбор технологии клиентской части

Для реализации пользовательского интерфейса рассматривались три основных подхода: библиотека React, фреймворк Angular и фреймворк Vue.js.

Angular предлагает полный корпоративный стек с мощной типизацией и строгой архитектурой, однако его избыточность и крутой порог вхождения замедляют разработку динамичных интерфейсов для MVP-версии проекта.

Vue.js отличается простотой и гибкостью, но его экосистема менее универсальна по сравнению с React, а поиск специалистов на рынке труда сложнее.

React был выбран как оптимальное решение благодаря компонентному подходу, высокой скорости рендеринга виртуального DOM и обширному сообществу. Возможность создания одностраничных приложений с динамическим обновлением интерфейса без полной перезагрузки страницы соответствует требованиям к интерактивности системы.

Выбор технологии серверной части

При определении языка программирования для серверной логики рассматривались три основных варианта: Node.js (JavaScript/TypeScript), Java (Spring Framework) и Python.

Асинхронная модель Node.js, основанная на обратных вызовах, сложнее в поддержке и отладке по сравнению с лаконичным синтаксисом `async/await` в современном Python. Кроме того, в python представлены более зрелые библиотеки для реализации алгоритмов распределения игроков.

Java со стеком Spring Boot является индустриальным стандартом для крупных корпоративных систем, однако, его использование связано с высокими временными затратами на разработку и необходимостью написания значительного объёма шаблонного кода. Для быстрой разработки Java избыточна.

Python был выбран как оптимальное решение, поскольку сочетает высокую скорость разработки (лаконичный код, динамическая типизация), богатую коллекцию библиотек для обработки данных и широкие возможности для асинхронного программирования.

После утверждения языка программирования был выполнен сравнительный анализ фреймворков, доступных для Python: FastAPI, Django и Flask.

Django — мощный фреймворк с большим набором встроенных компонентов, однако он тяжеловесен для асинхронной обработки запросов.

Flask — микрофреймворк, предоставляющий свободу в выборе расширений, но требует значительных усилий для добавления асинхронности и валидации данных.

FastAPI был выбран как фреймворк, обеспечивающий высокую производительность благодаря поддержке асинхронности, автоматической генерации OpenAPI-спецификации и встроенной валидации данных на основе Pydantic. Эти возможности позволят ускорить разработку REST-сервисов и обеспечить надежную обработку запросов при взаимодействии с клиентским приложением.

Выбор системы управления базами данных

Из требований к системе были выделены требования к системе управления базами данных:

- Открытый исходный код, отсутствие необходимости в лицензии.
- Наличие встроенного типа перечислений ENUM обеспечивает строгость и читаемость данных, ограничивая возможные значения статусов.
- Реляционная модель. Необходимы механизмы внешних ключей (FOREIGN KEY), проверочные ограничения (CHECK) и каскадные операции (ON DELETE CASCADE/SET NULL), которые гарантируют согласованность и непротиворечивость данных при любых манипуляциях.
- Производительность и масштабируемость за счёт индексирования по первичным ключам и потенциальным полям для поиска (например, username, email) обеспечит высокую скорость выполнения запросов даже при росте числа пользователей.
- Встроенные средства управления доступом и шифрования позволяют безопасно хранить конфиденциальные данные, такие как хэши паролей.

Были рассмотрены четыре популярные системы управления базами данных: PostgreSQL, MySQL, Microsoft SQL Server, SQLite и MongoDB.

MySQL и Microsoft SQL Server требуют наличие лицензии при использовании в коммерческих целях, поэтому могут быть недоступны. Встраиваемая в backend SQLite не подходит под некоторые критерии безопасности, а также ограничена в масштабировании физически. MongoDB не является реляционной, поэтому тоже не отвечает требованиям.

Для реализации базы данных проекта был выбран PostgreSQL — реляционная система управления базами данных с открытым исходным кодом, соответствующая требованиям проекта.

Схема взаимодействия и развертывания

Клиентское приложение React взаимодействует с сервером FastAPI посредством REST API по протоколу HTTP/HTTPS. Сервер обрабатывает запросы, выполняет бизнес-логику и обращается к базе данных PostgreSQL для чтения и сохранения информации.

Для создания защищённого соединения (подписывание сертификатов SSL), распределения нагрузки и маршрутизации будет использован Nginx.

Общая структура развертывания системы представлена на UML-диаграмме (рисунок 9).

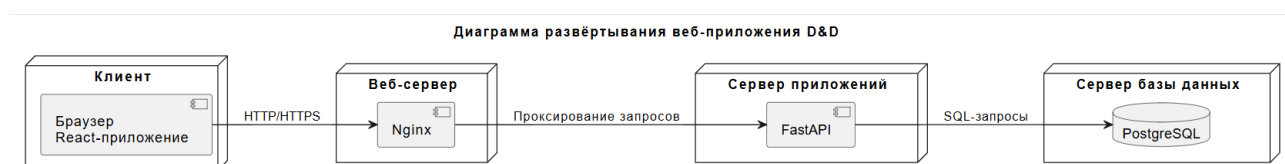


Рисунок 9 — Компонентная архитектура системы

1.2.4 Диаграмма классов и их связь (Сергей Пухарев)

В разрабатываемой системе было выделено несколько классов: User (пользователь), Moderator (модератор), Game (игра), Table (таблица) и Character (персонаж).

Класс User – основной класс пользователей. Он хранит логин пользователя, хэш его пароля, salt – добавочные данные для хэша, список игр, которые создал пользователь, персонажей пользователя, наличие прав модератора, приоритет аккаунта пользователя для участия в играх, дату создания аккаунта. Класс User обладает методом проверки пароля на корректность при попытке входа.

Класс Moderator – дочерний класс класса User. Представляет пользователя с правами модератора. Имеет методы модерации игр и персонажей.

Класс Game – это непосредственно сама игра, или же партия. Он хранит в себе статус игры (на проверке, отклонена, одобрена, закончилась), описание

игры, количество игроков, запланированное время проведения, дату создания игры. У класса Game есть метод смены статуса.

Класс Table – это таблица игроков, которая создаётся для записи на любую из игр. Эта таблица содержит список игроков, сортирует их по приоритету и автоматически выбирает игроков для игры, исходя из результата.

Класс Character – класс персонажей, которые создаются пользователями для игр. У каждого пользователя может быть несколько персонажей. Этот класс содержит статус персонажа (на проверке, одобрен, отклонён), имя персонажа, его уровень, короткое описание персонажа, полное описание персонажа, URL-ссылку на изображение с персонажем, пользователя-владельца персонажа, дату создания персонажа. У этого класса есть метод смены статуса.

На основе этих классов и их связи друг с другом была создана диаграмма классов в соответствии с правилами по самоучителю [3]. Она была выполнена в редакторе «draw.io» [4] и представлена на рисунке 10.

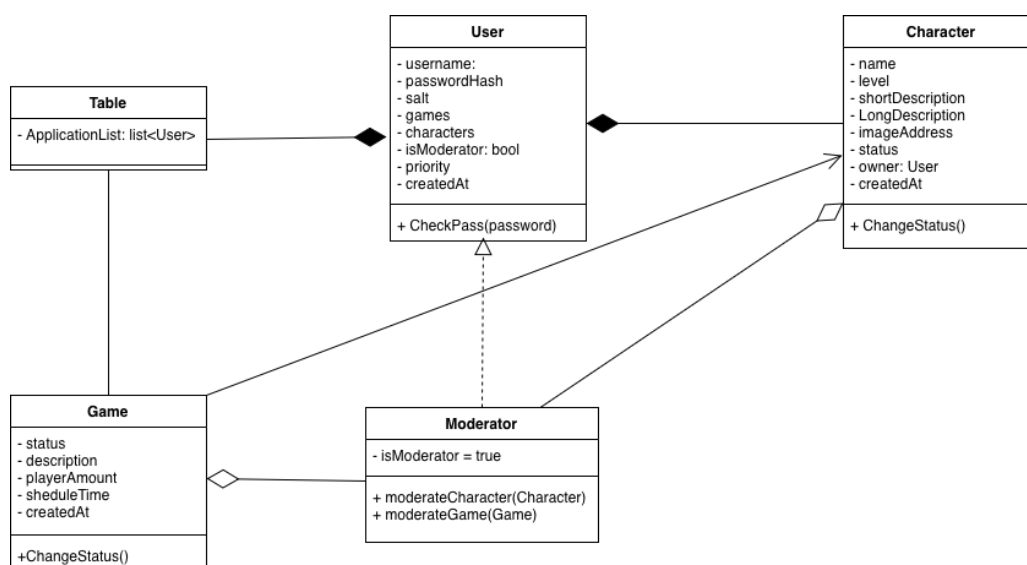


Рисунок 10 — Диаграмма классов

1.2.5 Диаграммы последовательности, деятельности и состояния (Сергей Пухарев)

На рисунке 11 представлена диаграмма последовательности процесса создания персонажа или игры пользователем.



Рисунок 11 — Диаграмма последовательности

На рисунке 12 представлена диаграмма деятельности процесса создания игры пользователем.



Рисунок 12 — Диаграмма деятельности

На рисунке 13 представлена диаграмма состояния процесса создания игры пользователем.

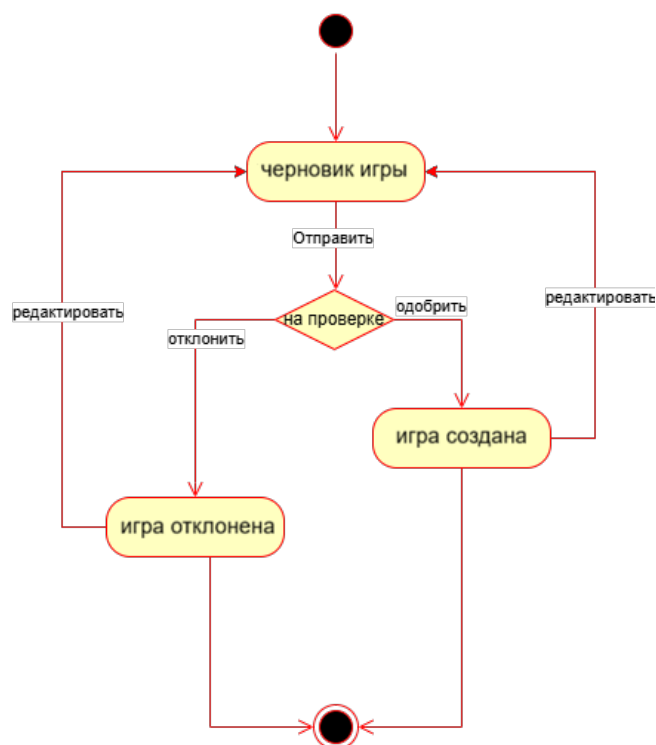


Рисунок 13 — Диаграмма состояния

1.2.6 Схема базы данных (Иван Сафронов)

Обоснование структуры и архитектуры базы данных

Проектируемая база данных является реляционной, что обусловлено необходимостью строгой структурированности данных, транзакционной целостности и поддержки сложных запросов с соединениями таблиц. Такая модель обеспечивает согласованность при параллельном доступе и позволяет реализовать бизнес-логику на уровне ограничений целостности.

В результате анализа предметной области выделены четыре основные сущности: пользователи (users), игровые персонажи (characters), игровые партии (parties) и заявки на участие (registrations). Сущность registrations является ассоциативной и реализует связь «многие-ко-многим» между characters и parties, что позволяет моделировать ситуацию, когда один

персонаж подаётся на несколько партий, а на одну партию претендуют множество персонажей.

Таблица «users» (id, username, password_hash, email, priority_score, created_at, is_moder) хранит учётные записи: аутентификационные данные, приоритетный скоринг для алгоритма справедливой записи, флаг модератора и метку регистрации.

Таблица «characters» (id, user_id, name, level, short_description, long_description, image_address, created_at, is_active) представляет персонажей: привязку к владельцу, уникальное имя, уровень, описания разной детализации, ссылку на изображение и флаг активности для «мягкого удаления».

Таблица «parties» (id, dm_id, title, description, min_players, max_players, status, schedule_date, schedule_time, location, image_address, min_player_level, max_player_level, created_at) описывает игровые сессии: мастера, заголовок и описание, ограничения по количеству и уровню участников, статус жизненного цикла, время и место проведения, обложку.

Таблица «registrations» (id, party_id, character_id, status, applied_at) фиксирует заявки: связь с партией и персонажем, решение по заявке и время подачи, используемое при равном приоритете.

На уровне схемы декларированы первичные ключи (id во всех таблицах), внешние ключи с правилами (для зависимых записей, доменные ограничения (диапазоны уровней 1–20, логическое согласование max_players не менее min_players и др.), уникальность (username, email в users; name в characters), а также ENUM-типы для статусов партии и заявок, исключающие невалидные значения.

Для производительности предполагается создание индексов по полю user_id в characters, составного индекса по (party_id, status) в registrations, индекса по status в parties и индекса по priority_score в users.

Структура находится в третьей нормальной форме: отсутствуют транзитивные зависимости, данные не дублируются, что минимизирует аномалии вставки, обновления и удаления. ER-диаграмма представлена на рисунке 14.

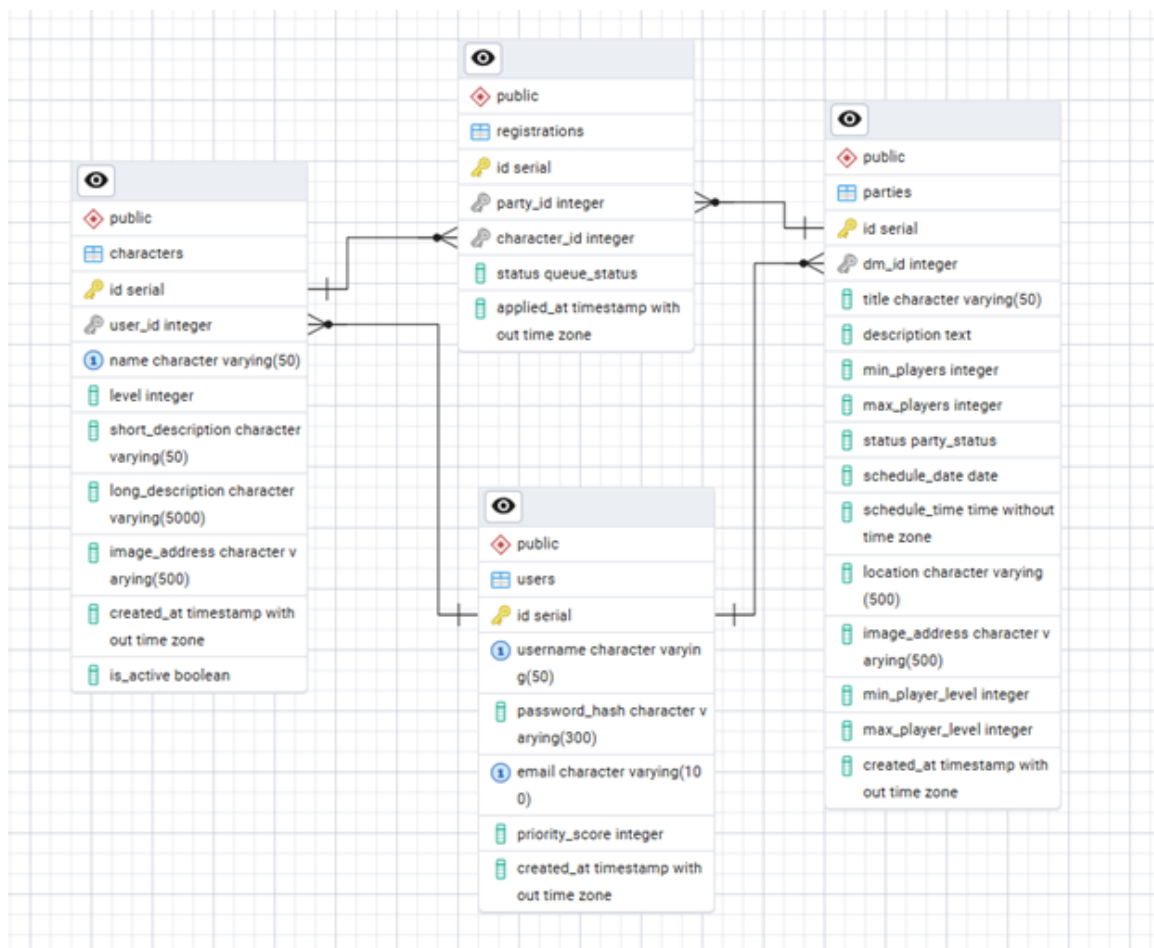


Рисунок 14 — ER-диаграмма базы данных

Предложенная архитектура полностью соответствует требованиям к системе регистрации на D&D-партии с приоритетным отбором и допускает расширение функционала без изменения ядра схемы.

1.2.7 Дизайн-прототип (Дмитрий Мартенс)

Для создания прототипа интерфейса платформы была выбрана среда дизайн-моделирования Figma [5]. Созданный прототип интерфейса основан на базовых принципах дизайн-моделирования:

- удобство пользования: интерфейс интуитивно понятен;
- визуальная иерархия: использованы разные размеры и начертания шрифтов, цвета и контрасты для управления вниманием пользователя, чтобы важные элементы были заметны;

- минимализм и простота: в интерфейсе отсутствуют лишние элементы, вызывающие визуальный шум, чтобы пользователь мог сосредоточиться на главном контенте, улучшая восприятие;
- согласованность: в интерфейсе использован единый стиль (шрифты, цвета, формы кнопок) для улучшения восприятия;
- типографика и читаемость: использованы понятные шрифты, достаточные отступы и цветовые контрасты;
- контраст и баланс: создана гармоничная композиция, которая привлекает внимание, но не перегружает интерфейс.

Обоснование и обзор интерфейса

В текущем виде прототип интерфейса содержит три страницы и четыре вариации. (Все рисунки и фотографии, которые используются в прототипе интерфейса взяты с бесплатного интернет-ресурса Freerik [6]) На рисунке 15 изображена страница регистрации пользователя на платформу:



Рисунок 15 — Страница регистрации

На данной странице пользователь создаёт аккаунт, под которым будет заходить на платформу. В интерфейсе присутствуют поля для ввода имени («nickname») и пароля («password»); checkbox («remember me?»), в котором пользователь отмечает, необходимо ли запомнить его данные в системе; кнопка «зарегистрироваться» («sign up»), по нажатию на которую регистрация завершается; кнопка «войти» («login») перенесёт пользователя на страницу входа в аккаунт, если у пользователя уже есть аккаунт.

На рисунке 16 изображена страница входа пользователя на платформу при наличии уже зарегистрированного на платформе аккаунта:

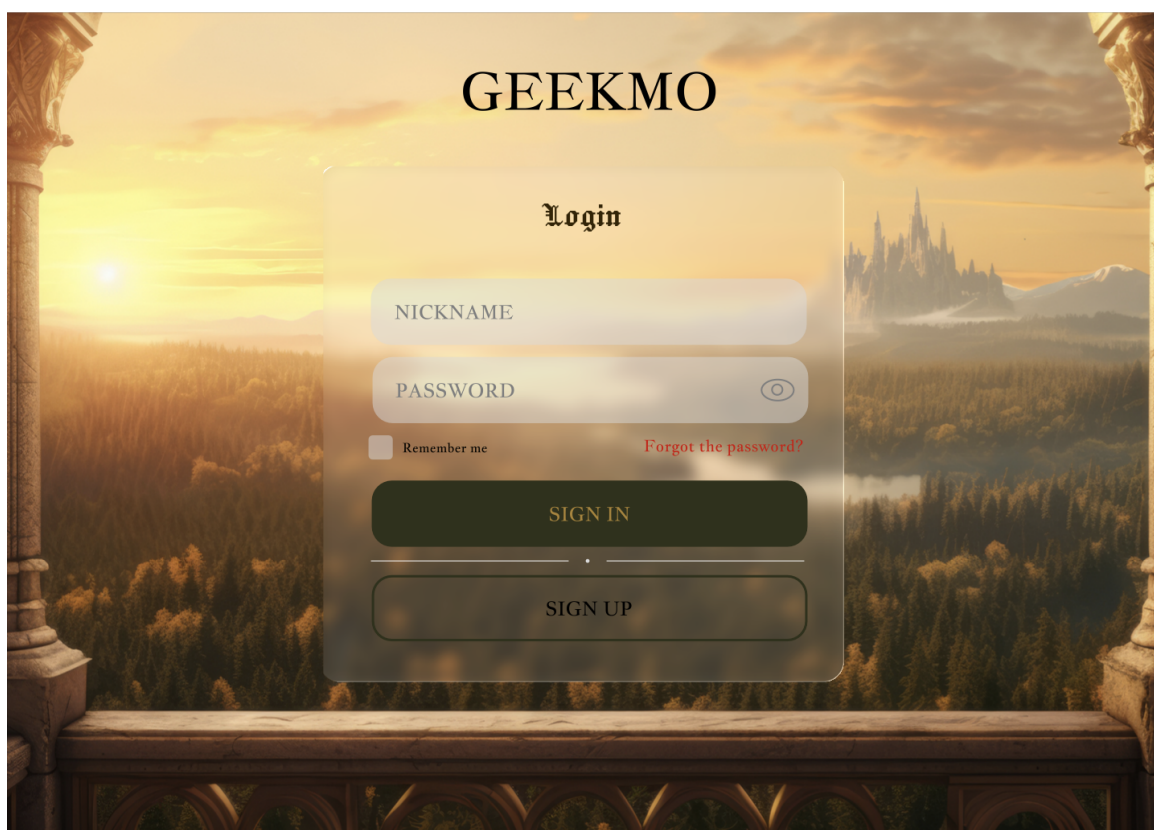


Рисунок 16 — Страница входа

На данной странице пользователь создаёт аккаунт, под которым будет заходить на платформу. В интерфейсе присутствуют поля для ввода имени («nickname») и пароля («password»); checkbox («remember me?»), в котором пользователь отмечает, необходимо ли запомнить его данные в системе; кнопка действия «Forgot the password?» на случай, если пользователь забыл пароль от аккаунта, и ему необходимо восстановить его; кнопка «войти» («sign in»), по нажатию на которую пользователь войдёт в систему; кнопка

«зарегистрироваться» («sign up») перенесёт пользователя на страницу регистрации, если у пользователя ещё нет зарегистрированного аккаунта на платформе.

На рисунке 17 изображена главная страница сервиса – лента открытыми партиями:



Рисунок 17 — Основная лента

В заголовке данной страницы находятся названия двух вкладок: «Партии» («Games») и «Профиль» («Profile»). Полоса контрастного цвета указывает, на какой вкладке находится пользователь. Переключение между вкладками осуществляется путём нажатия на её название.

В теле данной странице находится бесконечная обновляемая лента с игровыми партиями, созданными другими пользователями платформы. На данном рисунке представлен пример взаимодействия с одной из партий, опубликованной на платформе. Пользователь видит рисунок, загруженный создателем (мастером) партии при её создании, который визуально описывает сюжет компании. Над рисунком находится организационная

информация по проведению партии: название компании, имя мастера партии, дата проведения. В нижней части рисунка находится краткая информация о партии в стандартном режиме. В таком виде пользователю доступна информация об обязательных требованиях для проведения партии: количество игроков в отряде и минимальный уровень персонажа, который может принимать участие в игре.

Если пользователь наведёт курсор на блок с краткой информацией о партии, этот блок увеличится, открыв аннотацию по сюжету компании (рисунок 18). Если пользователь уведёт курсор с данного блока, он снова скроется.



Рисунок 18 — Дополнительная информация по партии

Ниже находятся кнопки действия: «Лист игроков» («Players list») и «Записаться» («Check in»).

При нажатии на кнопку «Лист игроков» («Players list») выдвигается таблица, содержащая информацию о пользователях, зарегистрировавшихся на игру (рисунок 19).



Рисунок 19 — Таблица «Лист игроков»

В открывшейся таблице находится имя пользователя («Player's name»), имя персонажа («Character»), уровень персонажа («Level») и приоритетные баллы пользователя («Priority»).

В нижней части таблицы находится кнопка действия, позволяющая развернуть таблицу целиком — по умолчанию показывается только первые четыре информационные строки.

При нажатии на кнопку «Лист игроков» («Players list») она загорается, сигнализируя, какое окно сейчас открыто. При повторном нажатии таблица скрывается.

При нажатии на кнопку действия «Записаться» («Check in») выдвигается окно выбора персонажа, под которым пользователь запишется на партию (рисунок 20).

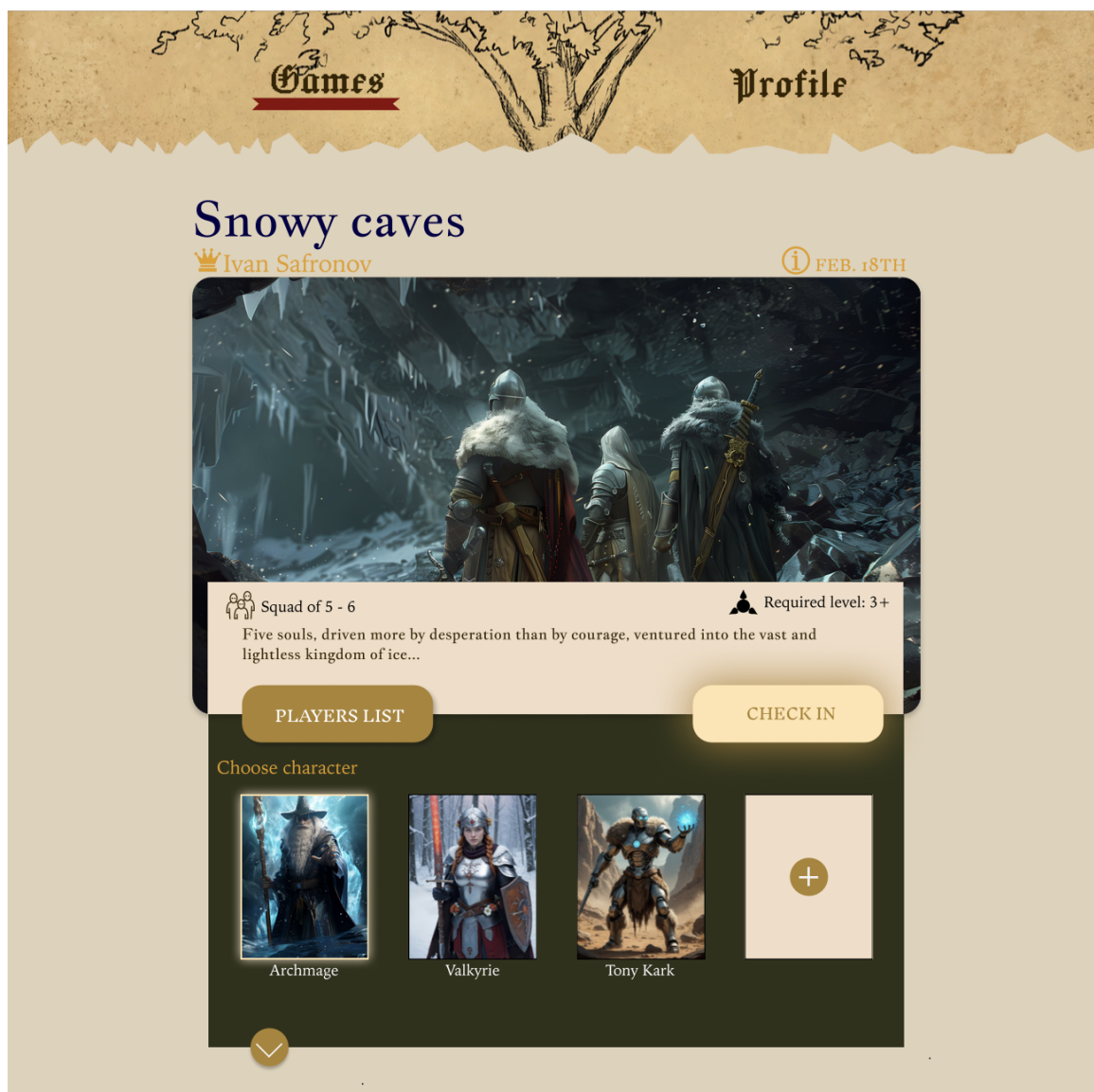


Рисунок 20 — Выбор персонажа

В окне находится список, в котором отображается аватар или облик персонажа и его имя. Тот персонаж, на которого нажмёт пользователь, будет записан на партию. Также из данного блока можно сразу переместиться на страницу создания персонажа в профиле.

Разработанный прототип интерфейса в своём нынешнем виде соответствует стандартам проектирования дизайна интерфейсов.

1.3 Выводы

За период практики были достигнуты следующие результаты:

- проанализирована предметная область;
- рассмотрены аналоги проекта;
- сформулированы требования к системе;
- составлена диаграмма прецедентов;
- спроектированы ключевые бизнес-процессы системы;
- определены технологии разработки;
- составлена диаграмма развёртывания;
- разработана база данных;
- создан прототип дизайна интерфейса.

ЗАКЛЮЧЕНИЕ

В процессе прохождения практики были пройдены этапы организации команды, анализа предметной области, разработки проекта системы и прототипирование.

Планируемые результаты практики достигнуты полностью.

На текущий момент начинается этап создания минимальной рабочей версии продукта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Fantasy Grounds LFG [Электронный ресурс]. – 2026. – URL: <https://www.fantasygrounds.com/> (дата обращения 06.02.2026).
2. Tabletop Wizard [Электронный ресурс]. – 2026. – URL: <https://www.rpgtablefinder.com/> (дата обращения 06.02.2026).
3. Леоненков А.В. Самоучитель UML. – 2-е изд., перераб. и доп. – СПб.: БХВ-Петербург, 2004. – 432с.
4. Flowchart Maker & Online Diagram Software – draw.io [Электронный ресурс]. – 2026. – URL: <https://app.diagrams.net/> (дата обращения 09.02.2026).
5. Figma [Электронный ресурс]. – 2026. – URL: <https://www.figma.com/> (дата обращения 09.02.2026).
6. Freepick [Электронный ресурс]. – 2026. – URL: <https://www.freepik.com> (дата обращения 09.02.2026).